PROJECT REPORT

Pocket Ledger

A private, offline expense tracker in Python

STUDENT	[Vishesh Mishra]
REGISTRATION NO.	[26BCE10955]
COURSE	[CSE 1021]
FACULTY	[Devraju S]
SUBMITTED	[30-09-2026]

CONTENTS

What's inside

01 Introduction 3

02 Problem Statement 3

03 Functional Requirements 4

04 Non-functional Requirements 5

05 System Architecture 6

06 Design Diagrams 7

07 Design Decisions and Rationale 12

08 Implementation Details 13

09 Screenshots and Results 14

10 Testing Approach 18

11 Challenges Faced 18

12 Learnings and Key Takeaways 19

13 Future Enhancements 19

14 References 19

SECTION 01

Introduction

Pocket Ledger is a personal expense tracker that runs in the terminal. It is built for people who want to understand where their money goes without creating accounts, installing large apps or sending financial data to anyone. Every expense is recorded with an amount, category, description and date, compared against monthly budgets, and turned into simple summaries. It applies core Python ideas (type-hinted functions, classes, file handling, exceptions, comprehensions, modular design) and adds tests, logging and documentation.

4	9	28	0
functional modules	Python modules	unit tests, all passing	external dependencies

SECTION 02

Problem Statement

> Most people know what they earn but not where it goes. Pocket Ledger is a small offline tool that records spending quickly, warns before a budget is crossed, and explains the month in plain reports.

Objectives

- Record, view, edit, delete and search expenses with strict input validation.
- Set monthly budgets per category and get OK, WARNING and OVER BUDGET status automatically.
- Provide a monthly summary, category chart, top expenses and monthly trend.
- Support CSV import and export with clear reporting of bad rows.
- Never lose or corrupt data, and keep a log of what the program does.

Scope and target users

IN SCOPE	OUT OF SCOPE	TARGET USERS
Single-user expense tracking, budgets and alerts, reports, CSV transfer, local JSON storage, command-line interface.	Multiple users, cloud sync, bank or UPI integration, income tracking, multiple currencies, graphical interface.	College students and young professionals who are comfortable running a Python program.

SECTION 03

Functional Requirements

Four modules, all driven from one numbered menu. Every module takes typed input, validates it and returns a clear message.

ID	MODULE	REQUIREMENT
FR1	Expenses	Add an expense with amount, category, description and date (date defaults to today).
FR2	Expenses	View all expenses in a table sorted by date.
FR3	Expenses	Update any field of an expense by ID; blank input keeps the old value.
FR4	Expenses	Delete an expense by ID after confirmation.
FR5	Expenses	Search by keyword, category, date range and amount range, alone or combined.
FR6	Budgets	Set or remove a monthly budget for a category.
FR7	Budgets	Show status per category: OK, WARNING (80% or more used) or OVER BUDGET.
FR8	Budgets	Show alerts automatically when the program starts.
FR9	Reports	Monthly summary: count, total, average and category share.
FR10	Reports	Text bar charts of spending by category and by month.
FR11	Reports	List the five most expensive expenses.
FR12	Import / export	Export all expenses to CSV.
FR13	Import / export	Import from CSV, skipping and reporting invalid rows.

How a session flows

On start the program loads the data file, recovers if it is corrupt, prints any budget alerts for the current month and shows the main menu. The user picks a module and answers the prompts. The input is validated, the operation runs, the file is saved and a result message is shown. Errors appear as messages and the user is returned to the menu; bad input never crashes the program.

SECTION 04

Non-functional Requirements

QUALITY	HOW IT IS MET
Reliability	Saves go to a temporary file and then replace the real file, so a crash cannot leave half a file. A corrupt file is renamed to *expenses.json.corrupt-timestamp* and the program starts clean instead of failing.
Security	All input passes through one validation module. Money is exact Decimal. CSV cells starting with =, +, - or @ are prefixed on export so spreadsheets do not run them as formulas. Data never leaves the machine. No eval.
Usability	Numbered menus, short prompts, friendly full-sentence messages, sensible defaults for date and month, confirmation before delete, alert summary at start-up.
Error handling	Validators raise ValueError with a clear message. The menu loop catches ValueError and OSError, prints 'An error occurred: ...', logs a warning and carries on.
Logging	Every change and every error is written with timestamp and level to logs/expense_tracker.log.
Maintainability	Small typed functions, one job per module, validation written once, and a test suite that guards refactoring.
Performance and efficiency	Standard library only, no background work. Data is loaded once and cached; linear scans stay instant for thousands of records.
Scalability	Managers only need load and save from the storage class, so JSON can be swapped for SQLite without touching menus, reports or validation.

SECTION 05

System Architecture

The application has three layers plus shared helpers. The presentation layer talks only to the business layer, the business layer holds every rule, and only the data layer touches the file. Validation and logging are used by all layers.

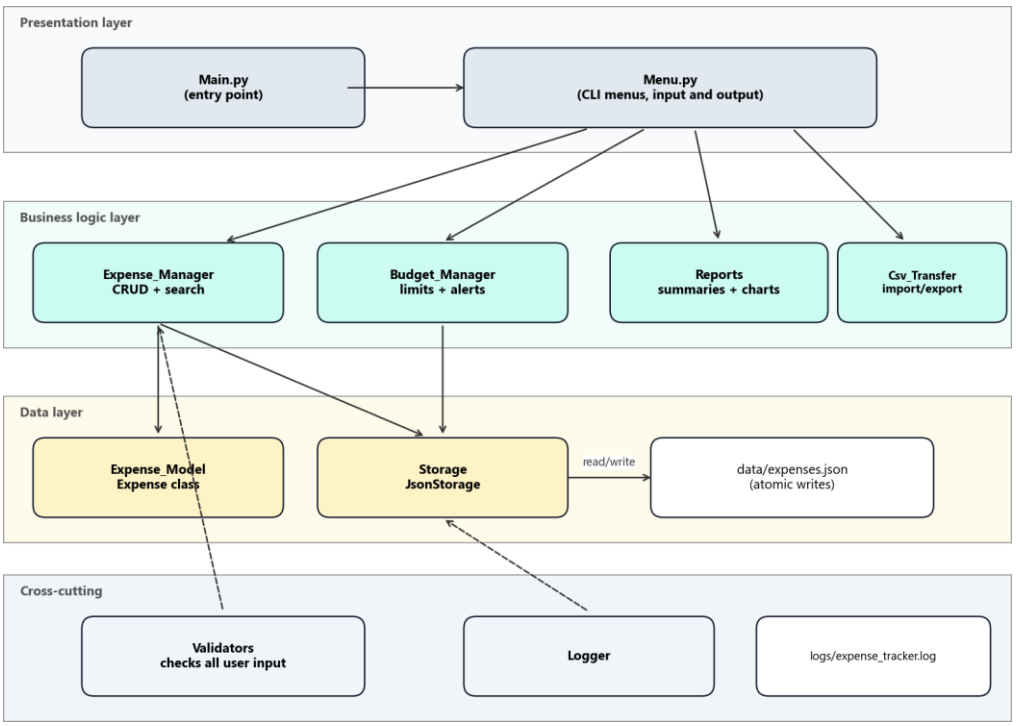


Figure 1. Layered architecture

FILE	RESPONSIBILITY
Main.py	Entry point; prints the result of mainMenu().
tracker/Menu.py	All menus and prompts; the only place that reads input or prints.
tracker/Expense_Manager.py	ExpenseManager: add, list, get, update, delete, search.
tracker/Budget_Manager.py	BudgetManager: budgets, monthly status and alerts.
tracker/Reports.py	Totals, category grouping, text charts, top expenses, trend.
tracker/Csv_Transfer.py	CSV export and import with formula protection.
tracker/Expense_Model.py, Storage.py	Expense class; JsonStorage with atomic saves and recovery.
tracker/Validators.py, Logger.py	Six validation functions; file logger.

SECTION 06

Design Diagrams

Use case diagram

One actor uses nine use cases. Adding, updating and importing all include input validation.

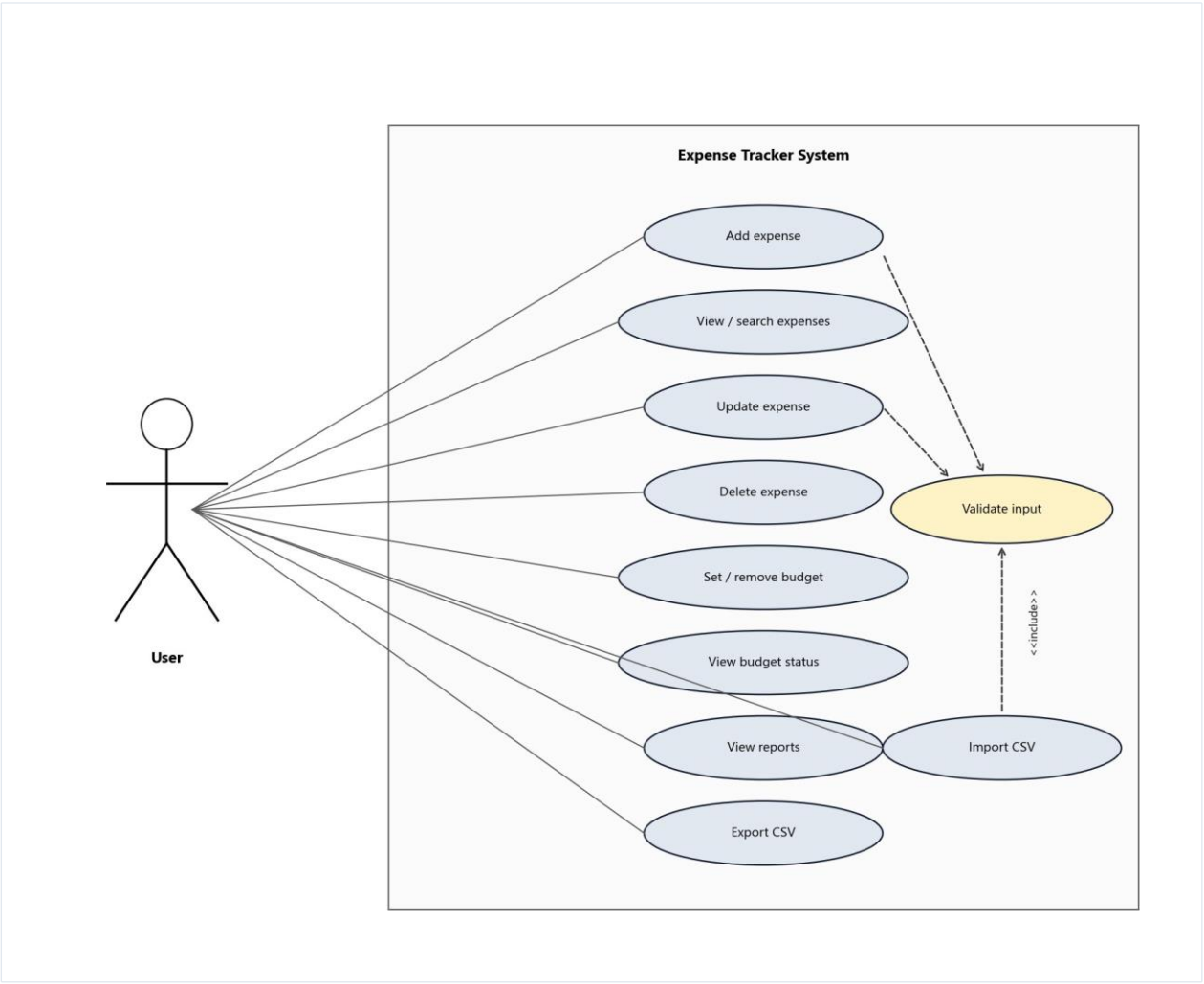


Figure 2. Use case diagram

Workflow diagram

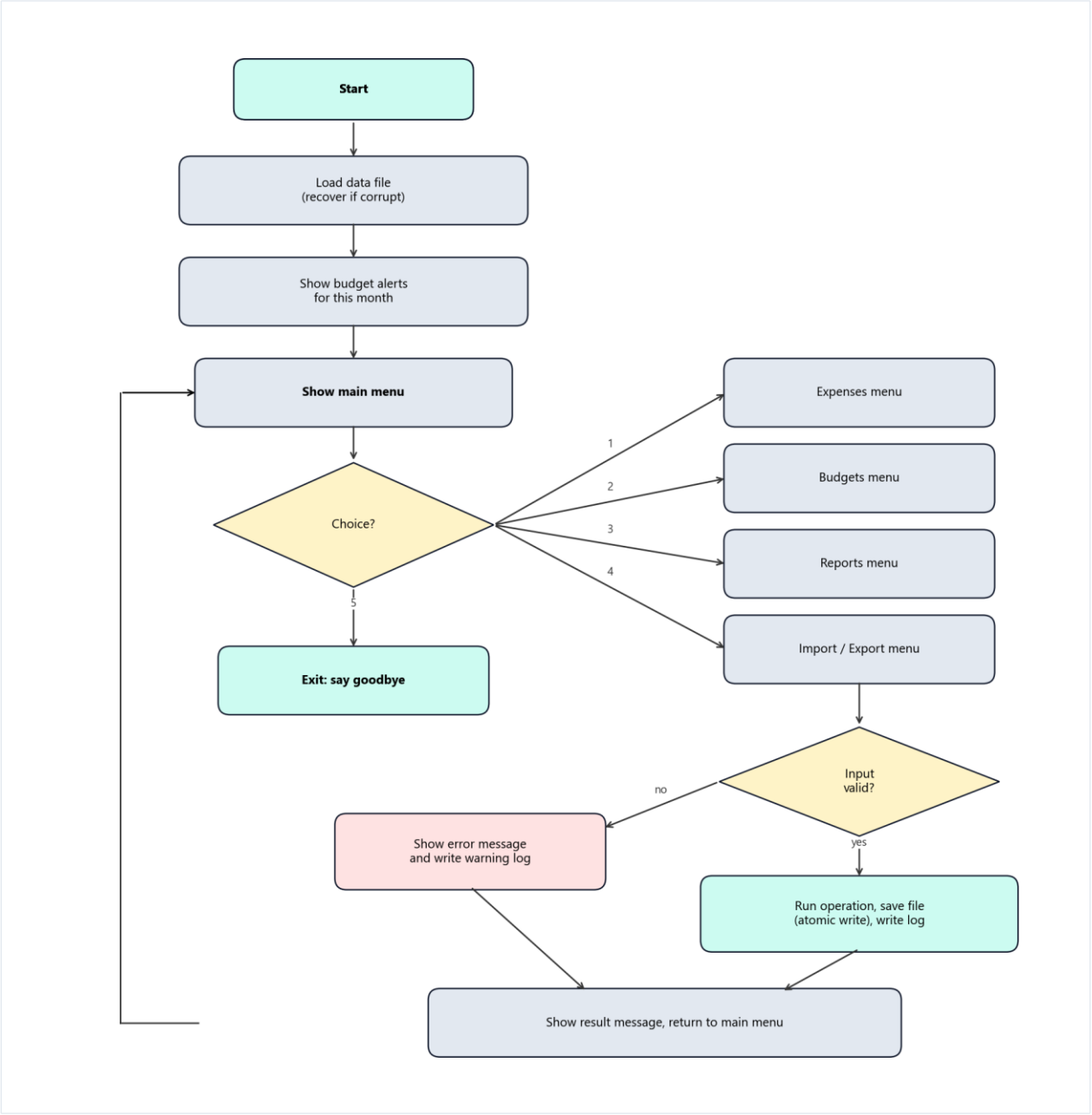


Figure 3. Workflow of a user session

Sequence diagram: add an expense

Validation, the atomic save, logging and the returned message, step by step.

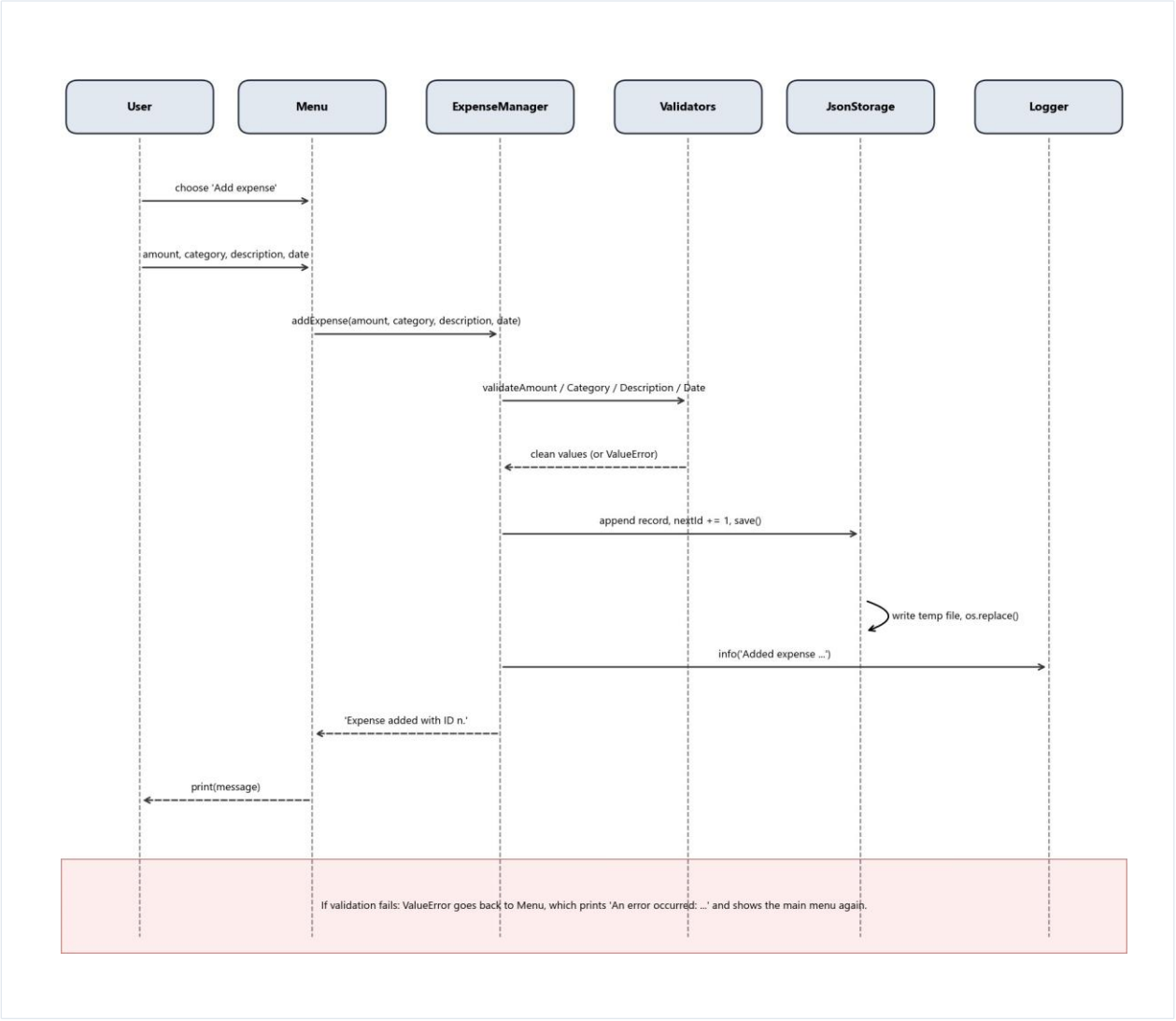


Figure 4. Sequence diagram

Class and component diagram

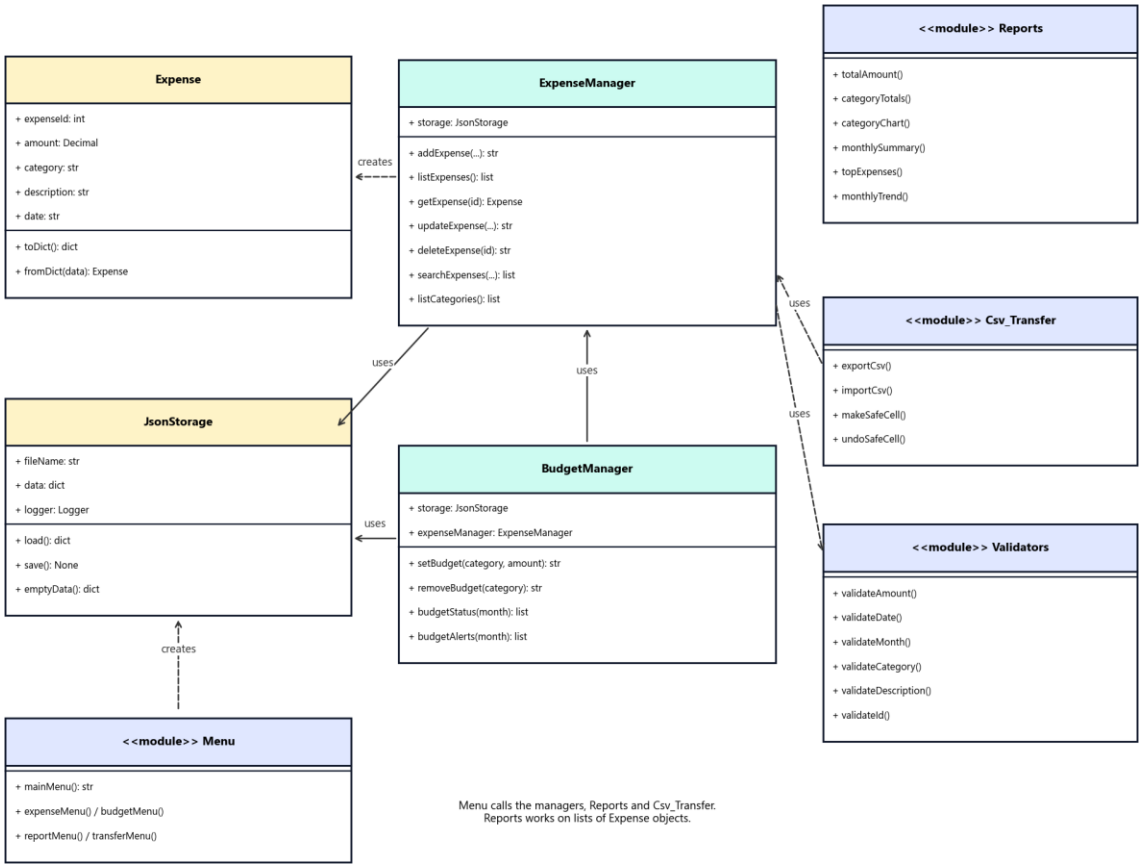


Figure 5. Classes and modules

ER diagram and storage schema

All data lives in one JSON file. Amounts are stored as strings so decimal values stay exact. The link between budgets and expenses is the category name.

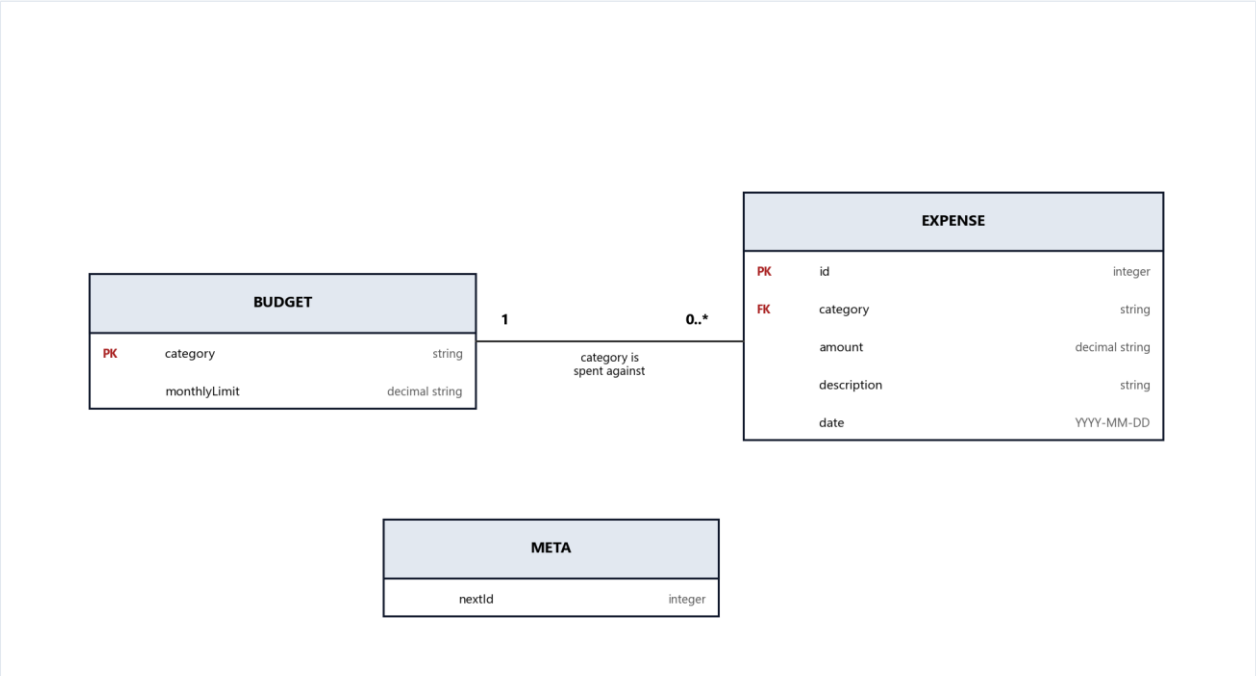


Figure 6. ER diagram

KEY	TYPE	EXAMPLE
nextId	integer	15
expenses[ ]	id, amount, category, description, date	14, "45.50", "Food", "Coffee with friends", "2026-09-29"
budgets{ }	category: limit (strings)	{"Food": "1000.00"}

SECTION 07

Design Decisions and Rationale

DECISION	WHY
Decimal, not float, for money	Floats give errors like 0.1 + 0.2 = 0.30000000000000004. Decimal keeps every total exact.
JSON file, not a database	Small single-user data. JSON is readable, needs no setup and is easy to back up. The storage class hides it, so a database can replace it.
Atomic save	Writing to a temp file and then os.replace means a crash or power cut cannot corrupt the existing file.
Managers return messages and raise ValueError	Logic can be tested without any input or printing; only Menu.py talks to the user.
One Validators module	Rules live in one place and protect manual entry, updates and CSV import alike.
Standard library only	Runs anywhere Python 3.10+ is installed; nothing for the user to install.
Text charts	Work in any terminal and paste into reports; no plotting library needed.
Future dates rejected	Catches typing mistakes such as the wrong year and keeps monthly totals honest.
Categories normalised to Title Case	'food', 'Food ' and 'FOOD' become one category, so budgets and reports always match.

SECTION 08

Implementation Details

The code follows one consistent style: camelCase names, type hints on every function, guard clauses that raise ValueError with full-sentence messages, early returns, f-strings and a numbered header comment above each function. Functions return results and messages; only the menu layer prints.

Concepts applied

CONCEPT	WHERE IT IS USED
Classes and objects	Expense, JsonStorage, ExpenseManager, BudgetManager
Exception handling	ValueError in validators, try / except in the menu loop, storage recovery
File handling	JSON and CSV with *with open(...)*, atomic writes, corrupt-file backup
Dictionaries and sets	Category totals, monthly totals, unique categories
Comprehensions, lambda, sorted	Search filters, sorting by date, amount and total
match / case	Every menu
Standard library	decimal, datetime, csv, json, logging, os, unittest

Key algorithms

- **Search.** Start from the sorted list and narrow it with one comprehension per filter that was supplied.
- **Budget status.** spent = sum of the category's expenses in the month; percent = spent / limit x 100. Over the limit is OVER BUDGET, 80% or more is WARNING, anything else is OK.
- **Category totals.** One pass builds a dictionary; the items are then sorted highest first.
- **Bar chart.** Bar length = amount / highest amount x 30 characters, never fewer than one.
- **Atomic save.** Write *file.tmp*, then os.replace onto the real file.

SECTION 09

Screenshots and Results

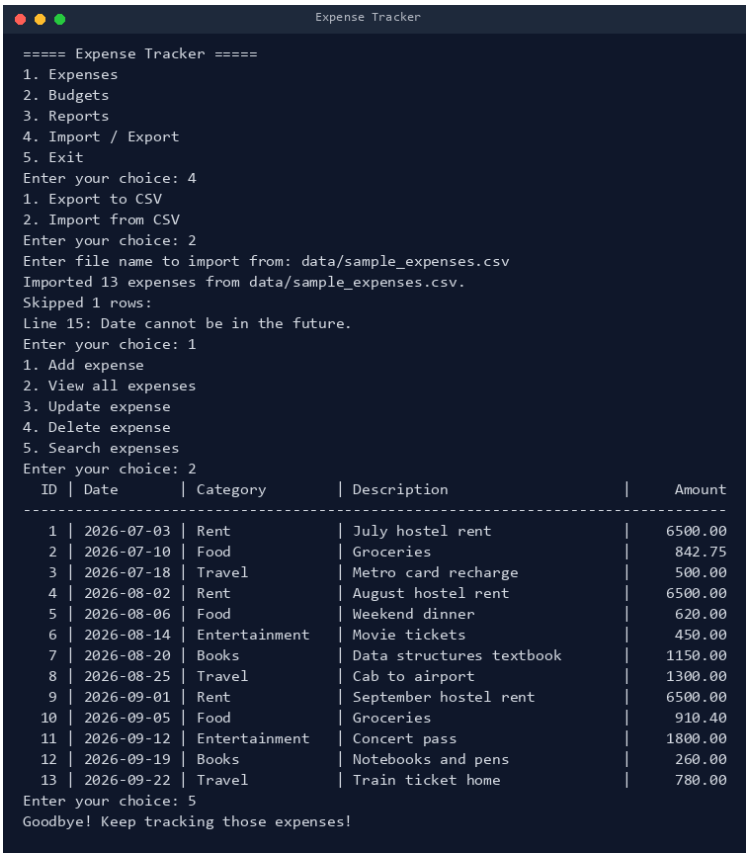
Captured by running the real program with scripted input on the sample data (docs/tools/make_screenshots.py). Each menu is shown once to keep the images short.

IMPORT

Load data from CSV

The sample file has one future-dated row. It is skipped and reported by line number while the rest are imported.

Figure 7

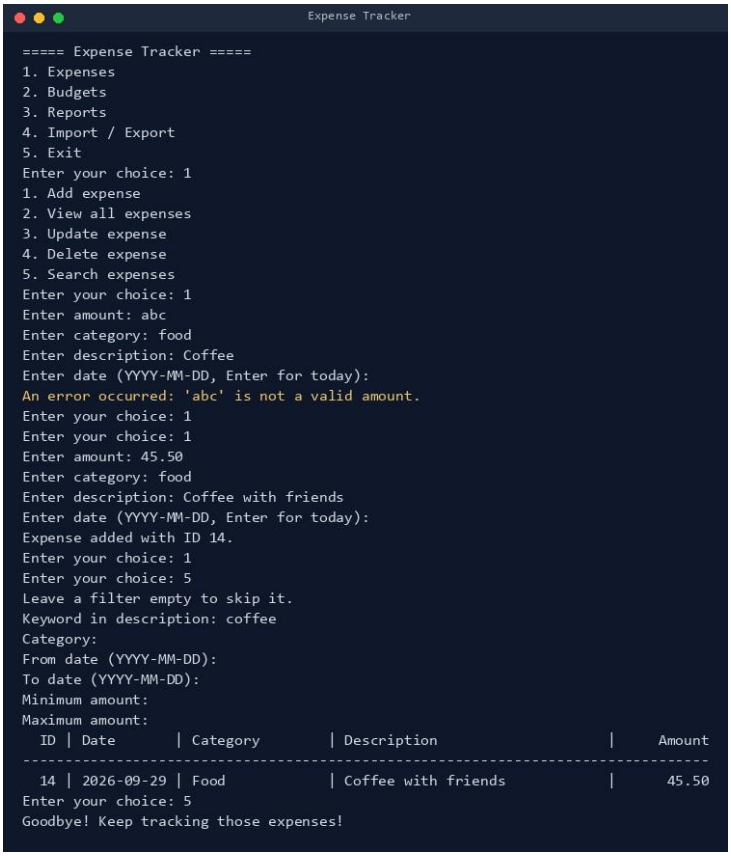


ADD AND SEARCH

Validation in action

A bad amount is rejected with a clear message and the program carries on. A valid entry is saved, then found by keyword.

Figure 8

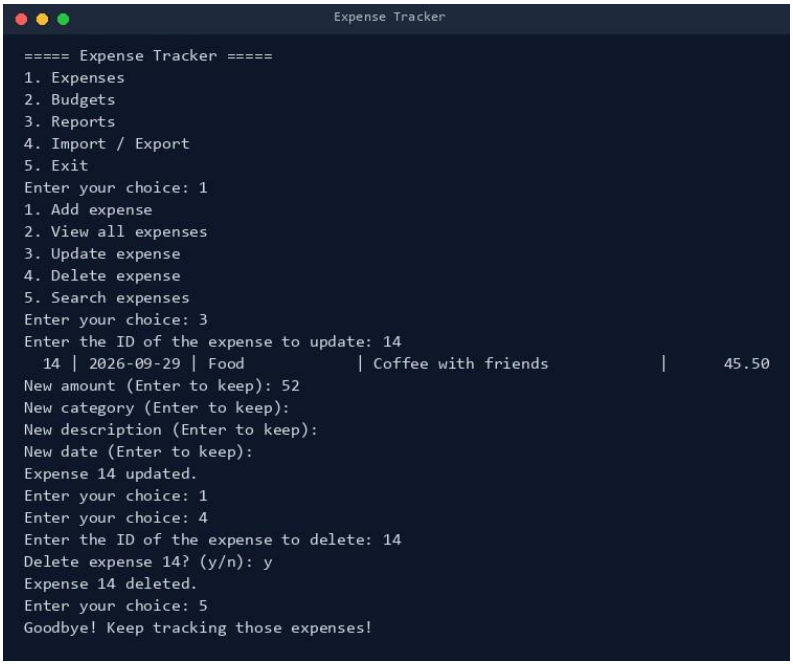


UPDATE AND DELETE

Edit safely

Blank answers keep the old values. Deleting asks for confirmation first.

Figure 9

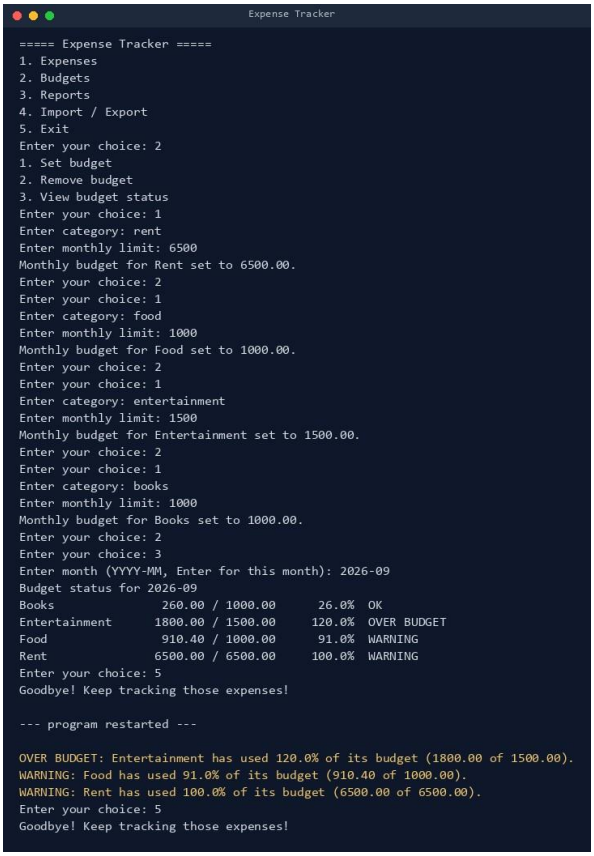


BUDGETS

Status and start-up alerts

Four budgets are set and September is checked. When the program restarts, the problem categories are shown immediately.

Figure 10

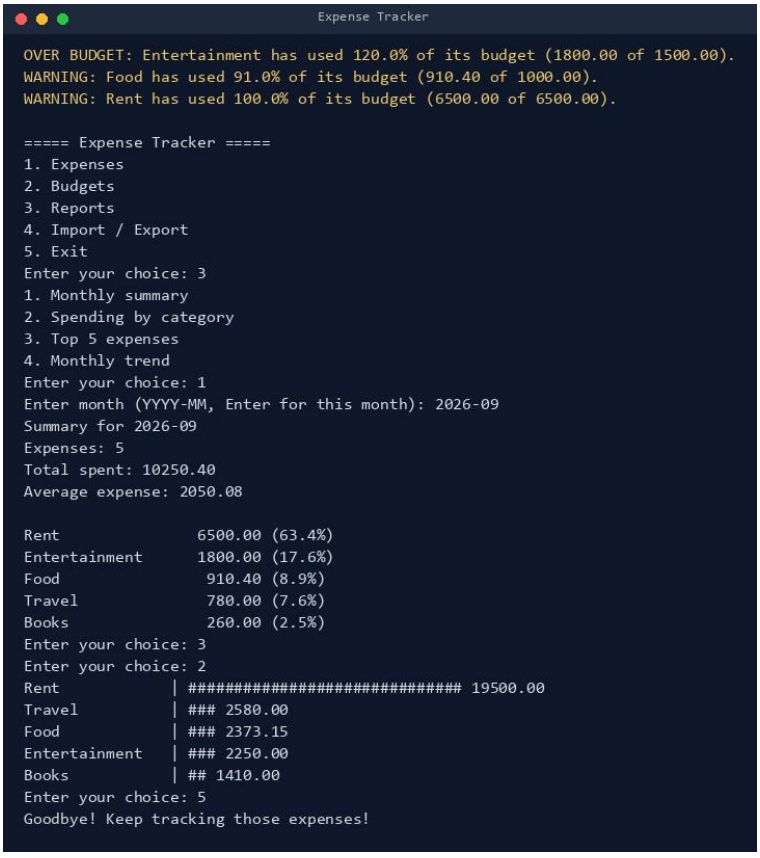


REPORTS

Summary and category chart

Totals, average and each category's share of the month, then a bar chart of spending by category.

Figure 11

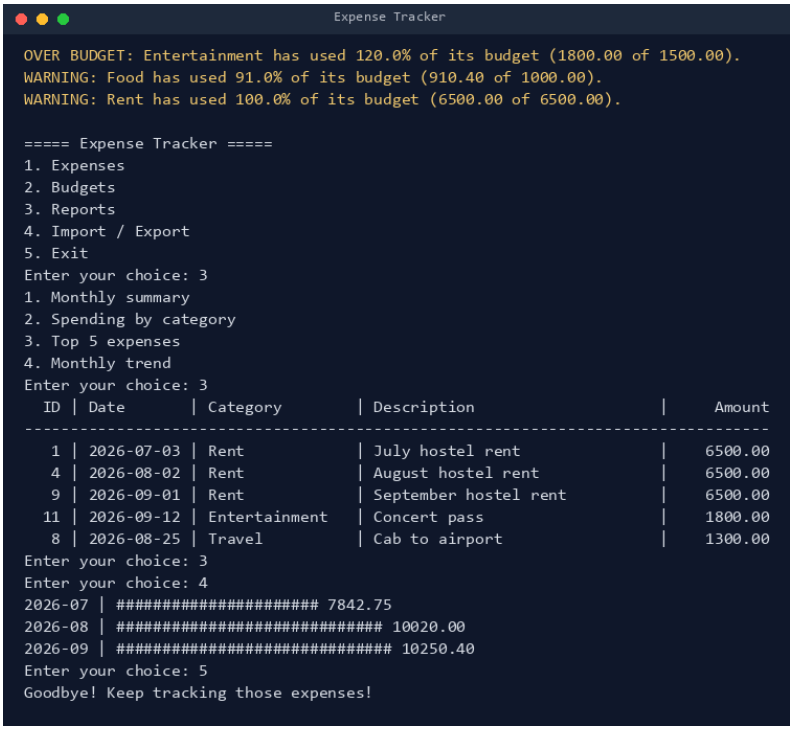


REPORTS

Top expenses and trend

The five largest expenses and a month-by-month bar chart.

Figure 12



EXPORT

Save to CSV

All expenses are written to a spreadsheet-friendly file.

Figure 13



SECTION 10

Testing Approach

Testing uses Python's built-in unittest. Every test creates its own temporary data folder, so tests never touch real data or each other.

TEST FILE	WHAT IT COVERS
test_validators.py	Amount rounding and rejection (text, zero, negative, NaN, Infinity, too large); date format and future dates; month, category, description and ID rules.
test_expense_manager.py	ID assignment, invalid input, persistence across restarts, partial updates, failed updates leaving data unchanged, delete, every search filter, corrupt-file recovery.
test_budget_and_reports.py	Budget labels and alerts, totals, monthly summary, top expenses, empty-data charts, CSV round trip with formula protection, bad-row skipping, wrong columns.

> **28 tests, all passing.** Run `python -m unittest discover -s tests -t .`. The full menu flow was also exercised end to end with scripted input, which confirmed that errors are shown and the program keeps running.

SECTION 11

Challenges Faced

- **Money precision.** Float arithmetic showed rounding errors, so everything moved to Decimal, stored as strings in JSON.
- **Lost messages.** Functions return messages instead of printing, so every caller had to be checked to make sure the message is shown.
- **Menu after errors.** Returning from an error handler would end the program, so the loop prints the error and continues.
- **Safe saving.** A plain write can corrupt the file if interrupted, which led to temp-file-and-replace and corrupt-file recovery.
- **CSV safety.** Text starting with = or @ can run as a formula in a spreadsheet, so export and import guard against it.
- **Month matching.** Comparing the YYYY-MM prefix of a date is simple but only safe because every date is validated to one format.

SECTION 12

Learnings and Key Takeaways

- Separating logic from input and output makes code far easier to test.
- Validating in one place is safer than checking wherever data is used.
- Early tests exposed edge cases, such as a failed update still changing data.
- Designing for failure (corrupt file, bad CSV row, bad input) matters as much as the normal path.
- Diagrams drawn during the work kept module boundaries clear.

SECTION 13

Future Enhancements

- Recurring expenses such as rent and subscriptions.
- Income tracking and a savings-rate report.
- SQLite storage behind the same storage interface.
- A graphical or web front end reusing the manager classes.
- Pie and line charts with matplotlib.
- Optional password protection or file encryption.
- Multiple currencies and PDF export of monthly reports.

SECTION 14

References

- Python Software Foundation. Python 3 documentation: decimal, json, csv, datetime, logging, unittest. https://docs.python.org/3/
- PEP 8, Style Guide for Python Code. https://peps.python.org/pep-0008/
- PEP 634, Structural Pattern Matching. https://peps.python.org/pep-0634/
- OWASP. CSV Injection. https://owasp.org/www-community/attacks/CSV_Injection
- VITyarthi. Build Your Own Project: General Project Instructions and Submission Guidelines.